

Assignment 3: Raw Socket Packet Analysis and Protocol Investigation

Name: JITUPAN MONDAL

Roll Number: EEB24023

Assigned Protocol: ICMP

Date: June 22, 2026

Cybersecurity Internship — Assignment 3

1 Objective

The objective of this assignment is to understand raw socket programming, IP packet structure, protocol headers, and packet analysis using Linux and Wireshark. A C program was developed using raw sockets to capture ICMP packets. Protocol-specific traffic was generated, captured, compared with Wireshark, and the packet header fields were analyzed in detail.

The assigned protocol for Roll Number EEB24023 (last digit: 3) is **ICMP** (Internet Control Message Protocol), which corresponds to IP Protocol Number 1.

2 System Configuration

The following system details were recorded using standard Linux commands before running the experiment.

Listing 1: System Information Commands and Output

```
$ uname -a
Linux jitupan-mondal-QEMU-Virtual-Machine 7.0.0-22-generic
#22-Ubuntu SMP PREEMPT_DYNAMIC Mon May 25 15:37:49 UTC 2026
aarch64 GNU/Linux

$ gcc --version
gcc (Ubuntu 15.2.0-16ubuntu1) 15.2.0
Copyright (C) 2025 Free Software Foundation, Inc.

$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536
    inet 127.0.0.1/8 scope host lo

2: enp0s1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500
    inet 192.168.64.2/24 brd 192.168.64.255 scope global dynamic
    enp0s1
```

Table 1: System Configuration Summary

Parameter	Value
Operating System	Ubuntu Linux (aarch64)
Kernel Version	7.0.0-22-generic
Architecture	aarch64 (ARM64 — Apple M4 via UTM)
GCC Version	15.2.0
Loopback Interface	lo — 127.0.0.1/8
Network Interface	enp0s1 — 192.168.64.2/24
Virtualization	QEMU (UTM on macOS)

3 Compile and Run Commands

Listing 2: Compilation and Execution

```
# Compile
gcc raw_capture.c -o raw_capture

# Run
sudo ./raw_capture

# Save output to file
sudo ./raw_capture | tee program_output.txt

# Generate ICMP traffic in a second terminal
ping -c 25 127.0.0.1
```

Root privileges are required because raw sockets operate below the transport layer and the Linux kernel restricts low-level network access to the root user.

4 Program Design

4.1 Overview

The program opens a raw socket using `socket(AF_INET, SOCK_RAW, IPPROTO_ICMP)`. This makes the kernel deliver only ICMP packets to the program. The program then repeatedly calls `recvfrom()`, which blocks until an ICMP packet arrives. Each received packet is stored in a byte buffer and parsed using C structs.

4.2 IP Header Parsing

The raw buffer starts with the IP header. The `struct iphdr` is overlaid on the buffer to access the IP fields directly. The IP header length is stored in the `ihl` field in units of 32-bit words, so it is multiplied by 4 to obtain the byte length.

4.3 ICMP Header Parsing

The ICMP header starts immediately after the IP header. Its location is calculated using:

```
struct icmphdr *icmp_header = (struct icmphdr *) (buffer +
    ip_header->ihl * 4);
```

The important ICMP fields are `type` and `code`. Type 8 with Code 0 indicates an Echo Request, while Type 0 with Code 0 indicates an Echo Reply.

4.4 Enhancement Field: Header Length

The additional IP header field chosen for enhancement is the **IP Header Length**. It is printed as `HEADER_LENGTH=%d bytes`. For standard ping traffic, this value remains 20 bytes because no IP options are present.

4.5 Source Code

Listing 3: *raw_capture.c* — *FullSourceCode*

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <netinet/ip.h>
#include <netinet/ip_icmp.h>
#include <arpa/inet.h>

#define PACKET_COUNT 20
#define BUFFER_SIZE 65536

int main() {
    int sock;
    unsigned char buffer[BUFFER_SIZE];
    struct sockaddr saddr;
    socklen_t saddr_len = sizeof(saddr);
    int packet_no = 0;

    printf("ROLL_NO=EEB24023\n");
    printf("ASSIGNED_PROTOCOL=ICMP\n\n");

    sock = socket(AF_INET, SOCK_RAW, IPPROTO_ICMP);
    if (sock < 0) { perror("socket() failed"); return 1; }

    printf("Listening for ICMP packets...\n\n");

    while (packet_no < PACKET_COUNT) {
        int data_size = recvfrom(sock, buffer, BUFFER_SIZE,
                                0, &saddr, &saddr_len);

        if (data_size < 0) {
            perror("recvfrom failed");
            close(sock);
            return 1;
        }

        struct iphdr *ip_header = (struct iphdr *)buffer;
        struct icmphdr *icmp_header =
            (struct icmphdr *)(buffer + ip_header->ihl * 4);

        struct in_addr src_addr, dst_addr;
        src_addr.s_addr = ip_header->saddr;
        dst_addr.s_addr = ip_header->daddr;

        packet_no++;

        printf("PACKET_NO=%d\n", packet_no);
    }
}
```

```
    printf("SRC_IP=%s\n",      inet_ntoa(src_addr));
    printf("DST_IP=%s\n",      inet_ntoa(dst_addr));
    printf("PROTOCOL=ICMP\n");
    printf("PROTOCOL_NO=%d\n",  ip_header->protocol);
    printf("TTL=%d\n",         ip_header->ttl);
    printf("PACKET_SIZE=%d\n",  data_size);
    printf("HEADER_LENGTH=%d bytes\n", ip_header->ihl * 4);
    printf("ICMP_TYPE=%d\n",    icmp_header->type);
    printf("ICMP_CODE=%d\n",    icmp_header->code);
    printf("\n");
}

close(sock);
printf("Done. Captured %d packets.\n", PACKET_COUNT);
return 0;
}
```

5 Traffic Generation

5.1 Method

ICMP traffic was generated using the standard `ping` command on the loopback interface. The loopback interface (`lo`, address `127.0.0.1`) was selected because it does not require an external network connection and is safe for a controlled lab environment.

Listing 4: Traffic Generation Command

```
ping -c 25 127.0.0.1
```

5.2 Observations

Each `ping` sends one ICMP Echo Request and receives one ICMP Echo Reply. Therefore, 25 pings generate approximately 50 ICMP packets. The raw socket program stops automatically after capturing 20 packets.

The captured interface was `lo`. All packets had:

- Source IP: `127.0.0.1`
- Destination IP: `127.0.0.1`
- TTL: 64
- Packet Size: 84 bytes

5.3 Screenshot: Traffic Generation

6 Experimental Results

The program successfully captured 20 ICMP packets. The output alternates between ICMP Type 8 (Echo Request) and ICMP Type 0 (Echo Reply), which is the expected

```

jitupan-mondal@jitupan-mondal-QEMU-Virtual-Machine: ~
PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data:
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.070 ms
64 bytes from 127.0.0.1: icmp_seq=2 ttl=64 time=0.116 ms
64 bytes from 127.0.0.1: icmp_seq=3 ttl=64 time=0.123 ms
64 bytes from 127.0.0.1: icmp_seq=4 ttl=64 time=0.075 ms
64 bytes from 127.0.0.1: icmp_seq=5 ttl=64 time=0.120 ms
64 bytes from 127.0.0.1: icmp_seq=6 ttl=64 time=0.131 ms
64 bytes from 127.0.0.1: icmp_seq=7 ttl=64 time=0.115 ms
64 bytes from 127.0.0.1: icmp_seq=8 ttl=64 time=0.195 ms
64 bytes from 127.0.0.1: icmp_seq=9 ttl=64 time=0.129 ms
64 bytes from 127.0.0.1: icmp_seq=10 ttl=64 time=0.130 ms
64 bytes from 127.0.0.1: icmp_seq=11 ttl=64 time=0.079 ms
64 bytes from 127.0.0.1: icmp_seq=12 ttl=64 time=0.226 ms
64 bytes from 127.0.0.1: icmp_seq=13 ttl=64 time=0.087 ms
64 bytes from 127.0.0.1: icmp_seq=14 ttl=64 time=0.160 ms
64 bytes from 127.0.0.1: icmp_seq=15 ttl=64 time=0.085 ms
64 bytes from 127.0.0.1: icmp_seq=16 ttl=64 time=0.078 ms
64 bytes from 127.0.0.1: icmp_seq=17 ttl=64 time=0.081 ms
64 bytes from 127.0.0.1: icmp_seq=18 ttl=64 time=0.085 ms
64 bytes from 127.0.0.1: icmp_seq=19 ttl=64 time=0.086 ms
64 bytes from 127.0.0.1: icmp_seq=20 ttl=64 time=0.092 ms

--- 127.0.0.1 ping statistics ---
20 packets transmitted, 20 received, 0% packet loss, time 19444ms
rtt min/avg/max/mdev = 0.070/0.113/0.226/0.040 ms
jitupan-mondal@jitupan-mondal-QEMU-Virtual-Machine: ~

```

Figure 1: Terminal output of `ping -c 25 127.0.0.1`

behavior for loopback ping traffic.

6.1 Program Output Screenshot

6.2 Sample Output (First 5 Packets)

Listing 5: `program_output.txt` — — *First5Packets*

```

ROLL_NO=EEB24023
ASSIGNED_PROTOCOL=ICMP

PACKET_NO=1
SRC_IP=127.0.0.1
DST_IP=127.0.0.1
PROTOCOL=ICMP
PROTOCOL_NO=1
TTL=64
PACKET_SIZE=84
HEADER_LENGTH=20 bytes
ICMP_TYPE=8
ICMP_CODE=0

PACKET_NO=2
SRC_IP=127.0.0.1
DST_IP=127.0.0.1
PROTOCOL=ICMP
PROTOCOL_NO=1
TTL=64

```

```
PACKET_SIZE=84
HEADER_LENGTH=20 bytes
ICMP_TYPE=0
ICMP_CODE=0

PACKET_NO=3
SRC_IP=127.0.0.1
DST_IP=127.0.0.1
PROTOCOL=ICMP
PROTOCOL_NO=1
TTL=64
PACKET_SIZE=84
HEADER_LENGTH=20 bytes
ICMP_TYPE=8
ICMP_CODE=0

PACKET_NO=4
SRC_IP=127.0.0.1
DST_IP=127.0.0.1
PROTOCOL=ICMP
PROTOCOL_NO=1
TTL=64
PACKET_SIZE=84
HEADER_LENGTH=20 bytes
ICMP_TYPE=0
ICMP_CODE=0

PACKET_NO=5
SRC_IP=127.0.0.1
DST_IP=127.0.0.1
PROTOCOL=ICMP
PROTOCOL_NO=1
TTL=64
PACKET_SIZE=84
HEADER_LENGTH=20 bytes
ICMP_TYPE=8
ICMP_CODE=0
```

7 Wireshark Verification

7.1 Capture Setup

Wireshark was configured to capture packets on the `lo` interface with the display filter `icmp`. The same `ping -c 25 127.0.0.1` command was used to generate the traffic. The capture was saved as `capture.pcapng`.

Table 2: Comparison of Raw Socket Program Output vs Wireshark

WS Frame	Prog Pkt	Source IP	Dest. IP	Proto.	TTL	Size *	Match?
1	1	127.0.0.1	127.0.0.1	ICMP	64	84 / 98	Yes
2	2	127.0.0.1	127.0.0.1	ICMP	64	84 / 98	Yes
3	3	127.0.0.1	127.0.0.1	ICMP	64	84 / 98	Yes
4	4	127.0.0.1	127.0.0.1	ICMP	64	84 / 98	Yes
5	5	127.0.0.1	127.0.0.1	ICMP	64	84 / 98	Yes

*Size: Program reports 84 bytes at IP layer. Wireshark reports 98 bytes including lower-layer framing.

7.2 Wireshark Screenshot

7.3 Comparison Table

7.4 Comparison Screenshot

7.5 Size Difference Explanation

The program reports `PACKET.SIZE=84` while Wireshark shows a larger frame length of 98 bytes. This is expected. The raw socket program captures packets starting from the IP header, so it measures 20 bytes of IP header + 8 bytes of ICMP header + 56 bytes of payload = 84 bytes. Wireshark captures at a lower layer and includes an additional 14 bytes of framing, which gives 98 bytes. Therefore, both values are correct.

8 Header Analysis

Three packets were selected for analysis: Packet 1, Packet 2, and Packet 3.

8.1 Why is the TTL value not zero?

TTL starts at 64 on Linux systems. It decreases only when a packet passes through a router. Since the experiment used the loopback interface, the packets never left the local machine and therefore did not cross any router. As a result, the observed TTL remained 64.

8.2 Why do packet sizes differ?

In this experiment, the packet size remained the same in the program output because the default ping payload size was constant. In general, packet sizes may differ if the payload size changes, if IP options are present, or if different ICMP message types are generated.

8.3 What information does the protocol field provide?

The protocol field in the IP header tells the operating system which upper-layer protocol should process the payload. In this assignment, the field value was 1, which corresponds to ICMP.

8.4 What could happen if the protocol field is modified?

If the protocol field is changed incorrectly, the receiving operating system may pass the payload to the wrong protocol handler. This would cause the packet to be misinterpreted or discarded.

8.5 What does the protocol-specific field reveal about the packet?

The ICMP Type and Code fields reveal the function of the packet. Type 8 Code 0 identifies an Echo Request, while Type 0 Code 0 identifies an Echo Reply. The alternating request-reply pattern confirmed correct ICMP communication.

9 Program Enhancement

9.1 Selected Field: IP Header Length

The chosen enhancement was the IP Header Length field.

```
printf("HEADER_LENGTH=%d bytes\n", ip_header->ihl * 4);
```

9.2 Purpose of the Field

The `ihl` field stores the header length in 32-bit words. Multiplying it by 4 converts the value into bytes. A standard IPv4 header without options has a length of 20 bytes.

9.3 Observed Values

Table 3: Observed Header Length Values

Packet No.	ihl Value	Header Length (bytes)
1	5	20
2	5	20
3	5	20
4	5	20
5	5	20

9.4 Analysis

The header length remained constant at 20 bytes for all observed packets. This shows that the ping packets used the standard IPv4 header format without optional fields.

10 Reflection Answers

1. Why are root privileges required for raw sockets?

Raw sockets allow direct access to IP packets and can also be used to craft custom packets. This is a privileged operation because it can be misused for sniffing or spoofing. Therefore, Linux requires root privileges.

2. How are raw sockets different from TCP/UDP sockets?

TCP and UDP sockets provide processed data after the kernel handles the transport-layer details. Raw sockets expose the packet contents directly, including the headers, and require the programmer to parse them manually.

3. State one advantage and one limitation of raw sockets.

Advantage: Raw sockets provide full visibility into packet headers and are useful for protocol analysis and security tools.

Limitation: They are low-level, require root access, and demand manual parsing of packet structures.

4. Mention one networking or cybersecurity application of raw sockets.

Raw sockets are used in packet analyzers, custom network diagnostic tools, and security tools such as port scanners and intrusion analysis systems.

11 Conclusion

This assignment successfully demonstrated the use of raw sockets in Linux for ICMP packet capture and analysis. A C program was written to capture 20 ICMP packets and display the required IP and ICMP header fields. Controlled traffic was generated safely using the loopback interface, and the captured output was verified with Wireshark.

The observations confirmed that the program parsed the packet structure correctly. The extra enhancement field, IP Header Length, remained constant at 20 bytes, which matched the standard IPv4 header format. The assignment provided practical understanding of raw sockets, IP headers, ICMP message structure, and packet verification using Wireshark.

The image displays three sequential screenshots of a terminal window running on a virtual machine named 'jitudan-mondal-QEMU-Virtual-Machine'. The terminal shows the execution of a program 'raw_capture.c' which listens for ICMP packets. The output displays details for four packets (PACKET_NO=1 to 4) in the first screenshot, and continues with packets 5 through 11 in the subsequent two screenshots. Each packet entry lists its source and destination IP addresses (127.0.0.1), protocol (ICMP), packet size (84 bytes), header length (20 bytes), ICMP type (8), and ICMP code (0). The terminal interface includes a sidebar with application icons and a top status bar showing system information like battery level and time.

```
jitudan-mondal@jitudan-mondal-QEMU-Virtual-Machine: ~  
jitudan-mondal@jitudan-mondal-QEMU-Virtual-Machine:~$ gcc raw_capture.c -o raw_capture  
jitudan-mondal@jitudan-mondal-QEMU-Virtual-Machine:~$ sudo ./raw_capture  
[sudo: authenticate] Password:  
ROLL_NO=EEB24023  
ASSIGNED_PROTOCOL=ICMP  
Listening for ICMP packets...  
  
PACKET_NO=1  
SRC_IP=127.0.0.1  
DST_IP=127.0.0.1  
PROTOCOL=ICMP  
PROTOCOL_NO=1  
TTL=64  
PACKET_SIZE=84  
HEADER_LENGTH=20 bytes  
ICMP_TYPE=8  
ICMP_CODE=0  
  
PACKET_NO=2  
SRC_IP=127.0.0.1  
DST_IP=127.0.0.1  
PROTOCOL=ICMP  
PROTOCOL_NO=1  
TTL=64  
PACKET_SIZE=84  
HEADER_LENGTH=20 bytes  
ICMP_TYPE=8  
ICMP_CODE=0  
  
PACKET_NO=3  
SRC_IP=127.0.0.1  
DST_IP=127.0.0.1  
PROTOCOL=ICMP  
PROTOCOL_NO=1  
TTL=64  
PACKET_SIZE=84  
HEADER_LENGTH=20 bytes  
ICMP_TYPE=8  
ICMP_CODE=0  
  
PACKET_NO=4  
SRC_IP=127.0.0.1  
DST_IP=127.0.0.1  
PROTOCOL=ICMP  
PROTOCOL_NO=1  
TTL=64  
PACKET_SIZE=84  
HEADER_LENGTH=20 bytes  
ICMP_TYPE=8  
ICMP_CODE=0  
  
PACKET_NO=5  
SRC_IP=127.0.0.1  
DST_IP=127.0.0.1  
PROTOCOL=ICMP  
PROTOCOL_NO=1  
TTL=64  
PACKET_SIZE=84  
HEADER_LENGTH=20 bytes  
ICMP_TYPE=8  
ICMP_CODE=0  
  
PACKET_NO=6  
SRC_IP=127.0.0.1  
DST_IP=127.0.0.1  
PROTOCOL=ICMP  
PROTOCOL_NO=1  
TTL=64  
PACKET_SIZE=84  
HEADER_LENGTH=20 bytes  
ICMP_TYPE=8  
ICMP_CODE=0  
  
PACKET_NO=7  
SRC_IP=127.0.0.1  
DST_IP=127.0.0.1  
PROTOCOL=ICMP  
PROTOCOL_NO=1  
TTL=64  
PACKET_SIZE=84  
HEADER_LENGTH=20 bytes  
ICMP_TYPE=8  
ICMP_CODE=0  
  
PACKET_NO=8  
SRC_IP=127.0.0.1  
DST_IP=127.0.0.1  
PROTOCOL=ICMP  
PROTOCOL_NO=1  
TTL=64  
PACKET_SIZE=84  
HEADER_LENGTH=20 bytes  
ICMP_TYPE=8  
ICMP_CODE=0  
  
PACKET_NO=9  
SRC_IP=127.0.0.1  
DST_IP=127.0.0.1  
PROTOCOL=ICMP  
PROTOCOL_NO=1  
TTL=64  
PACKET_SIZE=84  
HEADER_LENGTH=20 bytes  
ICMP_TYPE=8  
ICMP_CODE=0  
  
PACKET_NO=10  
SRC_IP=127.0.0.1  
DST_IP=127.0.0.1  
PROTOCOL=ICMP  
PROTOCOL_NO=1  
TTL=64  
PACKET_SIZE=84  
HEADER_LENGTH=20 bytes  
ICMP_TYPE=8  
ICMP_CODE=0  
  
PACKET_NO=11  
SRC_IP=127.0.0.1  
DST_IP=127.0.0.1  
PROTOCOL=ICMP  
PROTOCOL_NO=1  
TTL=64  
PACKET_SIZE=84  
HEADER_LENGTH=20 bytes  
ICMP_TYPE=8  
ICMP_CODE=0
```

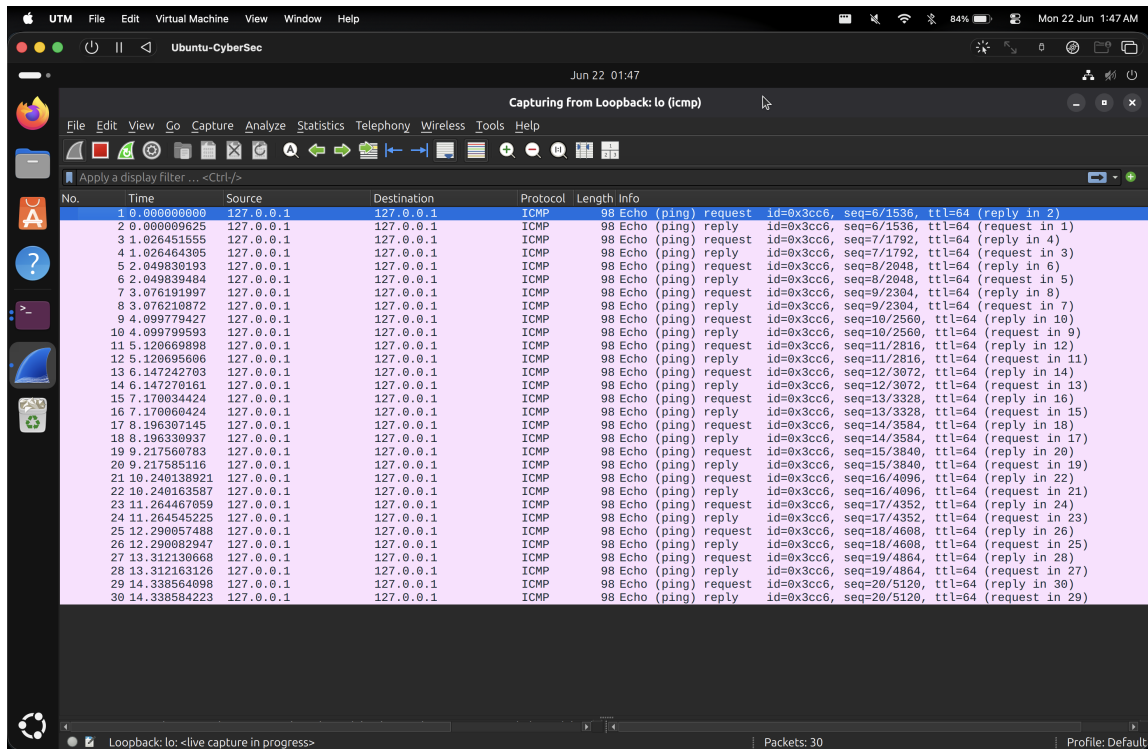


Figure 3: Wireshark capture on loopback interface with ICMP display filter

Wireshark vs Raw Socket Program — Packet Comparison

*Wireshark Length=98 includes 14-byte Ethernet header; program captures IP payload only (84 bytes)

Program Packet No.	Wireshark Frame No.	Source IP	Destination IP	Protocol	TTL	Packet Size (Program)	Packet Size (Wireshark)	Matched?
1	1	127.0.0.1	127.0.0.1	ICMP	64	84	98*	✓ Yes
2	2	127.0.0.1	127.0.0.1	ICMP	64	84	98*	✓ Yes
3	3	127.0.0.1	127.0.0.1	ICMP	64	84	98*	✓ Yes
4	4	127.0.0.1	127.0.0.1	ICMP	64	84	98*	✓ Yes
5	5	127.0.0.1	127.0.0.1	ICMP	64	84	98*	✓ Yes

Powered by perplexity

Figure 4: Comparison between raw socket output and Wireshark packets